

Urdu Basic Spelling Correction

ML, Deep Learning, and Transformer Model Report

Item	Details
Project task	Predict the corrected Urdu token from a noisy sentence and the marked wrong token.
Dataset	Urdu tweet pairs: original text and processed/corrected text.
Primary model in current code	Fine-tuned multilingual DistilBERT sequence classifier.
Current emphasis	Basic spelling and token-level correction patterns extracted from aligned tweet text.
Metrics reported	Accuracy, weighted precision, weighted recall, weighted F1-score, Top-3 accuracy, and loss.

Executive Summary

This report describes the current version of the Urdu spelling correction project. The current codebase is centered on token-level spelling correction: it cleans Urdu tweet text, aligns original and corrected token sequences, extracts one-to-one token replacements, and trains models to predict the corrected word from context.

The strongest current implementation is the reproducible DistilBERT script in `src/train_distilbert.py`. With the current saved run, it trains on 355 filtered examples across 17 correction classes and achieves 0.7183 test accuracy, 0.7103 weighted F1-score, and 0.8451 Top-3 accuracy.

- Classical TF-IDF models provide useful baselines; LinearSVC is the strongest classical model in the notebook results.
- Simple RNN underperforms because the dataset is small, noisy, and has many correction labels.
- LSTM improves over the Simple RNN by using gated memory, but remains below the current DistilBERT script result.
- The current DistilBERT workflow improves the earlier notebook transformer run through explicit wrong-token markers, class weighting, validation monitoring, and a reproducible training script.

1. Dataset and Preprocessing

The project uses `data/raw/UrduSpellDataset.csv`, which contains original Urdu tweets and their processed/corrected versions. The preprocessing script removes noisy social-media artifacts, keeps Urdu characters, normalizes whitespace, and standardizes common Urdu Unicode variants before token alignment.

Dataset statistic	Value
Total rows	4339
Missing processed tweets	35
Rows with any clean change	2658
All detected change records	6149

One-to-one replacement instances	1766
----------------------------------	------

The DistilBERT training script uses one-to-one token replacements by default. After filtering correction classes with fewer than 10 examples, the current training frame contains 355 examples and 17 target correction labels.

2. Modeling Task

The supervised learning task is correction classification. For each training example, the model receives a cleaned sentence and the suspected wrong token. The label is the corrected token. This means the model is not generating arbitrary text; it chooses one correction class from the label set learned during training.

Field	Meaning
Sentence	Cleaned original Urdu sentence from the tweet pair.
WrongWord	The token identified as changed during token alignment.
CorrectWord	The corrected target token from the processed tweet.
MarkedSentence	Sentence with [WRONG] and [WRONG] markers around the token being corrected.

3. Models and Architectures

3.1 Classical Machine Learning Models

The notebook evaluates classical text classifiers over TF-IDF features. The text representation combines sentence context with the wrong token, then extracts word n-gram features. These models are shallow, fast, and effective when correction patterns repeat in local context.

Model	Architecture / Method	Role
SGDClassifier	Linear classifier trained with stochastic gradient descent and logistic-style loss.	Fast discriminative baseline.
MultinomialNB	Probabilistic Naive Bayes classifier over sparse TF-IDF/count-like features.	Simple baseline for comparison.
LinearSVC	Linear support vector classifier over sparse TF-IDF n-gram features.	Strong classical margin-based baseline.

3.2 Deep Learning Models

The notebook also evaluates recurrent neural models. These models represent tokens with learned embeddings and process the sentence as a sequence rather than a bag of n-grams.

Model	Architecture	Expected advantage
Simple RNN	Embedding layer -> SimpleRNN(64) -> Dense(64, ReLU) -> Softmax.	Learns sequential patterns, but has limited memory and capacity.
LSTM	Embedding(128) -> LSTM(128, dropout/recurrent dropout) -> Dense(128) -> Dropout -> Softmax.	Handles longer context better through gated memory.

3.3 Transformer / Language Model: DistilBERT

The current main model is distilbert-base-multilingual-cased from Hugging Face. DistilBERT is a compact transformer encoder language model. In this project it is fine-tuned as a sequence classification model, not used as a generative chatbot or external API.

- Input representation: the wrong token is surrounded by [WRONG] and [WRONG] markers inside the sentence.
- Tokenizer input: MarkedSentence is passed as the first sequence and WrongWord as the paired second sequence.
- Output layer: a classification head predicts one of the filtered CorrectWord labels.
- Optimization: AdamW, learning rate 2e-5, weight decay 0.01, linear warmup schedule, gradient clipping, and label smoothing.
- Training controls: class-weighted cross-entropy, validation monitoring, early stopping patience, saved model artifacts, metrics, and test predictions.

4. Experimental Results

The table below combines the recorded notebook results with the current script-based DistilBERT result saved in outputs/distilbert/metrics.csv. The current DistilBERT run is the best Top-1 and Top-3 result in the project folder.

Model	Accuracy	Precision	Recall	F1-score	Top-3
SGDClassifier	0.4531	0.4443	0.4531	0.4197	0.6380
MultinomialNB	0.1094	0.1092	0.1094	0.0800	0.1823
LinearSVC	0.4661	0.4839	0.4661	0.4536	0.7318
Simple RNN	0.1484	0.1426	0.1484	0.1369	N/R
LSTM	0.4387	0.3890	0.4387	0.3965	0.5375
DistilBERT notebook	0.3443	0.2354	0.3443	0.2491	0.5246
DistilBERT current script	0.7183	0.7218	0.7183	0.7103	0.8451

4.1 Current DistilBERT Training Details

Current DistilBERT item	Value
Base model	distilbert-base-multilingual-cased
Filtered training examples	355
Correction classes	17
Train / validation / test rows	241 / 43 / 71
Epochs completed in saved history	11
Best validation accuracy	0.8605
Best validation F1-score at best-val epoch	0.8462
Test accuracy	0.7183
Weighted F1-score	0.7103

Top-3 accuracy	0.8451
Test loss	1.5897

5. Comparative Analysis

Classical ML comparison: LinearSVC is the strongest classical model because sparse TF-IDF n-grams capture repeated local spelling and token correction patterns. SGDClassifier is close behind, while Multinomial Naive Bayes performs poorly because its independence assumptions are too restrictive for contextual token correction.

Deep learning comparison: the Simple RNN has the weakest neural result because the dataset is small after label filtering and the label space is still broad. LSTM improves substantially because gated memory helps preserve context, but it does not exceed the strongest classical baseline or the current DistilBERT script.

Transformer comparison: the earlier notebook DistilBERT experiment underperformed the LSTM and LinearSVC baselines. The current script-based DistilBERT is much stronger because the wrong word is explicitly marked, class imbalance is handled with weighted loss, training runs for more epochs, and validation monitoring selects the best model state.

Top-3 behavior: the current DistilBERT Top-3 accuracy of 0.8451 shows that even when the first prediction is wrong, the correct spelling often appears among the top ranked suggestions. This is useful for an interactive GUI because the system can display multiple candidate corrections.

6. Demo Interface and Inference

The Streamlit app in app.py loads the saved DistilBERT model and lets a user enter an Urdu sentence. The app scans each Urdu token internally, marks one token at a time as the suspected error, asks the classifier for correction probabilities, and displays the strongest correction candidate.

Component	Purpose
app.py	Streamlit GUI for entering sentences and viewing predictions.
src/predict_correction.py	Inference utilities: load model, build token candidates, rank predictions, print CLI report.
outputs/distilbert/model/	Saved Hugging Face model and tokenizer.
outputs/distilbert/label_classes.json	CorrectWord label set used by the classifier.
outputs/distilbert/test_predictions.csv	Saved test predictions and confidence scores.

7. Limitations

- The current model predicts only labels that survived class-frequency filtering, so rare corrections are outside the output vocabulary.
- The dataset is noisy social-media Urdu, so results may not generalize to formal Urdu without additional validation.
- The model depends on a candidate wrong token being supplied or scanned; it is not a dedicated token-level error detector.
- The notebook and script results are not all from identical train/test splits, so comparisons should be interpreted as project-level experimental comparisons.

- Some Urdu rendering in terminals can look broken even when the model and GUI handle full words correctly; Word/GUI output should use Unicode-capable fonts.

8. Conclusion

The current project version is a solid Urdu basic spelling correction system built from token-level tweet corrections. Classical TF-IDF models establish competitive baselines, but the current reproducible DistilBERT training script is the strongest recorded model, reaching 0.7183 accuracy and 0.8451 Top-3 accuracy on the saved test split. The Streamlit GUI provides a practical demonstration layer by scanning words internally and showing the most confident correction suggestions.

Appendix: Reproducibility

Step	Command / Output
Install dependencies	<code>pip install -r requirements.txt</code>
Generate processed CSV files	<code>python src/scan_real_world_errors.py</code>
Check DistilBERT data split	<code>python src/train_distilbert.py --dry-run</code>
Train current DistilBERT model	<code>python src/train_distilbert.py</code>
Run CLI prediction	<code>python src/predict_correction.py "<Urdu sentence>" --no-curved</code>
Launch GUI demo	<code>python -m streamlit run app.py</code>
Saved metrics	<code>outputs/distilbert/metrics.csv, training_history.csv, test_predictions.csv</code>